

What is HTML5?

HTML5 is the latest version of HTML used to **structure content on web pages**. It introduces:

- Semantic tags
- Multimedia support
- Improved forms
- Browser storage
- Geolocation API

HTML5 Basic Structure

```
<!DOCTYPE html>

<html>

<head>

  <meta charset="UTF-8">

  <title>My Page</title>

</head>

<body>

  <h1>Hello World</h1>

</body>

</html>
```

2. Browser, BOM and DOM

Browser

Software used to display web pages (Chrome, Edge, Firefox)

BOM (Browser Object Model)

Represents browser components like:

- window
- navigator
- location
- history

DOM (Document Object Model)

Represents the HTML document as a tree structure.
Used by JavaScript to access and modify elements.

Example:

```
document.getElementById("demo").innerHTML = "Hello";
```

Inline vs Block Elements

Inline elements

Take only required width

Examples: , <a>,

Block elements

Take full width

Examples: <div>, <p>, <h1>

Hyperlink

```
<a href="https://example.com">Visit</a>
```

Navigation tag

```
<nav>
  <a href="#home">Home</a>
  <a href="#about">About</a>
</nav>
```

Reference to intermediate section

```
<a href="#contact">Go to Contact</a>
<h2 id="contact">Contact Section</h2>
```

Example:

```
<button onclick="alert('Clicked')">Click</button>
```

Web Forms 2.0

Important HTML5 form features:

- <input>
- <output>
- placeholder
- autofocus
- required

Example:

```
<input type="text" placeholder="Enter name" autofocus required>
```

localStorage

Data remains even after browser is closed

```
localStorage.setItem("name", "Bhanu");
localStorage.getItem("name");
localStorage.removeItem("name");
```

sessionStorage

- Data removed when tab/session ends

```
sessionStorage.setItem("user", "abc");
```

Difference

- **localStorage** → permanent until deleted
- **sessionStorage** → only for current session

:

3) JAVASCRIPT SHORT NOTES

What is JavaScript?

JavaScript is a scripting language used to make web pages **interactive and dynamic**.

JavaScript Setup

Can be written:

- inside <script> tag
- in external .js file

```
<script src="app.js"></script>
```

JavaScript Basics

Variables

```
let name = "Bhanu";  
const pi = 3.14;  
var age = 20;
```

Data Types

- Number
- String
- Boolean
- Null
- Undefined
- Object
- Array

DOM Manipulation

```
document.getElementById("demo").innerHTML = "Hello";
```

Event Handling

```
document.getElementById("btn").onclick = function(){  
  alert("Clicked");  
}
```

Arrays and Array Methods

```
let arr = [10,20,30];
```

Common methods

- push()
- pop()
- shift()
- unshift()
- map()
- filter()
- forEach()

Asynchronous JavaScript

Callback

A function passed into another function to run later.

Callback Hell

Too many nested callbacks making code hard to read.

Promise

Represents a future result.

```
let p = new Promise((resolve, reject)=>{  
  resolve("Done");  
});
```

async / await

```
async function getData(){  
  let response = await fetch(url);  
}
```

ES6+ Features

let and const

- let → can be changed
- const → cannot be reassigned

Template Literals

```
let name = "Bhanu";  
console.log(`Hello ${name}`);
```

Destructuring

```
let [a,b] = [10,20];
```

Rest Operator

```
function sum(...nums){}
```

Spread Operator

```
let a = [1,2];  
let b = [...a,3];
```

Default Parameters

```
function greet(name="Guest"){}
```

Fetch API

```
fetch("https://api.example.com/data")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

Debugging and Testing

- console.log()
- browser developer tools
- breakpoints

JavaScript Frameworks and Libraries

Examples:

- React
- Angular
- Vue
- jQuery

4) BOOTSTRAP 5 SHORT NOTES

What is Bootstrap?

Bootstrap is a **front-end CSS framework** used to build **responsive, mobile-first web pages quickly**.

Bootstrap Setup

Bootstrap can be added using CDN:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css" rel="stylesheet">
```

Bootstrap Grid System

Bootstrap grid has **12 columns**.

Example:

```
<div class="container">
  <div class="row">
    <div class="col-6">A</div>
    <div class="col-6">B</div>
  </div>
</div>
```

Important classes

- container
- row
- col
- col-md-6
- col-lg-4

Responsive Grid Layout

Used to create layouts for different screen sizes.

Example:

```
<div class="col-sm-12 col-md-6 col-lg-4">Box</div>
```

Alignment and Reordering

Bootstrap provides utility classes:

- justify-content-center
- align-items-center
- order-1
- order-2

Responsive Flexbox Utilities

- d-flex
- justify-content-between
- align-items-center
- flex-column

Bootstrap Components

Typography

Used to style text.

Examples:

- text-center
- fw-bold
- lead

Forms

```
<input class="form-control">
```

Buttons

```
<button class="btn btn-primary">Save</button>
```

Navbar

Navigation bar component.

Cards

```
<div class="card">  
  <div class="card-body">Hello</div>  
</div>
```

Media Objects

Used to align image and text together.

Bootstrap Utilities and Helpers

Spacing

- m-3 → margin
- p-2 → padding
- mt-3, mb-2

Colors and Backgrounds

- text-danger
- bg-primary

Display and Visibility

- d-none
- d-block

Borders, Shadows, Rounded Corners

- border
- shadow
- rounded

Positioning Utilities

- position-relative
- position-absolute

Advanced Bootstrap 5 Features

Bootstrap Icons

Used to add icons to pages.

JavaScript Plugins

Bootstrap provides plugins for:

- modal
- dropdown
- carousel
- collapse

Customization with Sass

Bootstrap can be customized using Sass variables and mixins.

5) JQUERY SHORT NOTES

What is jQuery?

jQuery is a **lightweight JavaScript library** that simplifies:

- DOM manipulation
- event handling
- animations
- AJAX

Features of jQuery

- Write less code
- Easy DOM manipulation
- Cross-browser support
- Chainable methods
- Simplifies events and AJAX

Including jQuery

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

The \$() Function

Used to select HTML elements.

Examples:

```
$("#id")
$(".class")
$("p")
```

6) ANSI SQL USING MYSQL

What is SQL?

SQL (Structured Query Language) is used to **store, retrieve, and manipulate data** in relational databases.

ANSI SQL

ANSI SQL is the standard form of SQL that provides common rules for database operations.

MySQL

MySQL is a popular relational database management system.

SQL Command Types

- **DDL** → CREATE, ALTER, DROP
- **DML** → INSERT, UPDATE, DELETE
- **DQL** → SELECT
- **DCL** → GRANT, REVOKE
- **TCL** → COMMIT, ROLLBACK

SELECT Statement

```
SELECT * FROM student;  
SELECT name, age FROM student;
```

WHERE Clause

```
SELECT * FROM student WHERE age > 18;
```

Used for filtering rows.

ORDER BY

```
SELECT * FROM student ORDER BY age;  
SELECT * FROM student ORDER BY age DESC;
```

Used for sorting data.

Filtering and Sorting Data

Using AND / OR

```
SELECT * FROM student  
WHERE age > 18 AND city='Guntur';  
SELECT * FROM student  
WHERE city='Guntur' OR city='Vijayawada';
```

Aggregate Functions

Used on groups of values:

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

Example:

```
SELECT COUNT(*) FROM student;  
SELECT AVG(age) FROM student;
```

GROUP BY

```
SELECT dept, COUNT(*)  
FROM employee  
GROUP BY dept;
```

Groups rows based on a column.

```
HAVING
SELECT dept, COUNT(*)
FROM employee
GROUP BY dept
HAVING COUNT(*) > 2;
```

Used to filter grouped results.

Difference

- **WHERE** → filters rows before grouping
- **HAVING** → filters grouped results after grouping

JOINS

INNER JOIN

Returns matching rows from both tables.

```
SELECT s.name, d.dept_name
FROM student s
INNER JOIN dept d ON s.dept_id = d.id;
```

LEFT JOIN

RIGHT JOIN

SELF JOIN

CROSS JOIN

Subqueries

A query inside another query.

```
SELECT name
FROM student
WHERE age > (SELECT AVG(age) FROM student);
```

```
INSERT
INSERT INTO student(id, name, age)
VALUES(1, 'Bhanu', 20);
```

```
UPDATE
UPDATE student
SET age = 21
WHERE id = 1;
```

```
DELETE
DELETE FROM student
WHERE id = 1;
```

```
CREATE TABLE
CREATE TABLE student(
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
```

```
ALTER TABLE
ALTER TABLE student ADD city VARCHAR(20);
```

```
DROP TABLE
DROP TABLE student;
```

Indexes and Constraints

Primary Key

Foreign Key

UNIQUE

CHECK

Example:

```
age INT CHECK(age >= 18)
```

Index

Used to improve search performance.

```
CREATE INDEX idx_name ON student(name);
```

7) C#

Introduction to C# and .NET

- **C#** is an object-oriented programming language developed by Microsoft.
- **.NET** is a platform for building applications such as desktop, web, and enterprise apps.

First C# Program
using System;

```
class Program
{
    static void Main()
    {
        Console.WriteLine("Hello World");
    }
}
```

```
}  
}
```

Variables and Data Types

Common data types:

- int
- float
- double
- char
- string
- bool

Example:

```
int age = 20;  
string name = "Bhanu";
```

Constants

```
const double PI = 3.14;
```

Value Type vs Reference Type

Value Types

Store actual value.

Examples:

- int
- float
- char
- bool
- struct

Reference Types

Store memory reference/address.

Examples:

- string
- array
- class
- object

Type Inference

```
var x = 10;  
var name = "Bhanu";
```

Control Flow Statements

if

```
if(age >= 18)
    Console.WriteLine("Adult");
```

switch

```
switch(day)
{
    case 1:
        Console.WriteLine("Monday");
        break;
    default:
        Console.WriteLine("Invalid");
        break;
}
```

Loops

```
for
for(int i=0; i<5; i++) { }
while
while(i < 5) { }
do-while
do { } while(i < 5);
foreach
foreach(var item in arr)
{
    Console.WriteLine(item);
}
```

Pattern Matching

Used with `is` and `switch` to check types or values in a more powerful way.

Functions and Methods

```
static int Add(int a, int b)
{
    return a + b;
}
```

Parameters

ref

Passes value by reference.

out

Used to return values from a method.

in

Passes read-only reference.

Method Overloading

Same method name with different parameters.

```
int Add(int a, int b)
double Add(double a, double b)
```

Local Functions

Functions defined inside another method.

OOP Concepts

Class and Object

```
class Student
{
    public string Name;
}
Student s = new Student();
```

Constructor

Special method called when object is created.

```
class Student
{
    public Student()
    {
        Console.WriteLine("Constructor called");
    }
}
```

Access Modifiers

- public
- private
- protected
- internal

Encapsulation and Properties

Property

```
class Student
{
    public string Name { get; set; }
}
```

Auto-implemented property

Uses get; set; without writing explicit fields.

init-only setter

Can be assigned only during object creation.

Inheritance

Allows one class to inherit from another.

```
class Animal { }  
class Dog : Animal { }
```

Polymorphism

Allows methods to behave differently.

Method Overriding

```
class Animal  
{  
    public virtual void Sound(){  
    }  
}  
  
class Dog : Animal  
{  
    public override void Sound(){  
    }  
}
```

Exception Handling

```
try  
{  
    int x = 10 / 0;  
}  
catch(Exception ex)  
{  
    Console.WriteLine(ex.Message);  
}  
finally  
{  
    Console.WriteLine("Done");  
}
```

Custom Exceptions

User-defined exception classes can be created.

Nullable Types and Null Safety

Nullable value type

```
int? age = null;
```

Null-coalescing operator

```
string name = input ?? "Guest";
```

Null-conditional operator

```
obj?.Method();
```

Collections

List

```
List<int> nums = new List<int>();
```

Dictionary

```
Dictionary<int, string> d = new Dictionary<int, string>();
```

Queue

FIFO

Stack

LIFO

LINQ

Language Integrated Query used for filtering and transforming data.

```
var result = nums.Where(x => x > 10).ToList();
```

Tuples

```
var student = (1, "Bhanu");
```

Used to store multiple values together.

Asynchronous Programming

```
async Task GetData()
```

```
{  
    await Task.Delay(1000);  
}
```

Uses:

- Task
- async
- await

File Handling and I/O

Read file

```
string text = File.ReadAllText("a.txt");
```

Write file

```
File.WriteAllText("a.txt", "Hello");
```

Multithreading

Used to execute multiple tasks concurrently.

Important ideas:

- Thread
- Task
- concurrency
- synchronization

8) ADO.NET

What is ADO.NET?

ADO.NET is a .NET data access technology used to **connect applications with databases**.

Used for:

- retrieving data
- inserting data
- updating data
- deleting data

Core ADO.NET Objects

1. Connection

Used to connect to database.

Example: SqlConnection

```
SqlConnection con = new SqlConnection("connection_string");
con.Open();
con.Close();
```

2. Command

Used to execute SQL commands.

Example: SqlCommand

```
SqlCommand cmd = new SqlCommand("select * from student", con);
```

3. DataReader

Used in connected architecture.

Reads data row by row.

```
SqlDataReader dr = cmd.ExecuteReader();
while(dr.Read())
{
    Console.WriteLine(dr[0]);
}
```

4. DataAdapter

Used to fill DataSet or DataTable.

```
SqlDataAdapter da = new SqlDataAdapter(cmd);
```

5. DataSet

In-memory collection of tables used in disconnected architecture.

6. DataTable

Represents a single table in memory.

Connected Architecture

Uses:

- Connection
- Command
- DataReader

Connection remains open while reading data.

Disconnected Architecture

Uses:

- DataAdapter
- DataSet
- DataTable

Data is loaded into memory, and connection can be closed.

Insert using ADO.NET

```
SqlCommand cmd = new SqlCommand("insert into student values(1,'Bhanu')", con);  
cmd.ExecuteNonQuery();
```

Update using ADO.NET

```
SqlCommand cmd = new SqlCommand("update student set name='Vaishnavi' where id=1", con);  
cmd.ExecuteNonQuery();
```

Delete using ADO.NET

```
SqlCommand cmd = new SqlCommand("delete from student where id=1", con);  
cmd.ExecuteNonQuery();
```

Select using ADO.NET

```
SqlCommand cmd = new SqlCommand("select * from student", con);  
SqlDataReader dr = cmd.ExecuteReader();
```

Execute Methods

ExecuteReader()

Used for SELECT queries.

ExecuteNonQuery()

Used for INSERT, UPDATE, DELETE.

ExecuteScalar()

Used to return a single value such as count, max, etc.